

BRAC's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 4

Roll no: 28 PRN No: 12411933 Name of Student: Pragati Rakhunde

Subject Name : Deep Learning

1. Problem Statement

Design and implement a Convolutional Neural Network (CNN) for image classification using a tomato leaf disease dataset. The objective is to train a CNN model that can automatically identify the disease category of a tomato leaf image.

The dataset contains images belonging to 10 classes, including healthy tomato leaves and different tomato diseases. The trained model is evaluated using validation accuracy, loss, classification report, and confusion matrix. A Streamlit-based web application is also developed to allow users to upload a tomato leaf image and obtain the predicted disease class with its confidence score.

2. Objective

- To understand the concept of Convolutional Neural Networks.
- To preprocess tomato leaf images for CNN training.
- To build a CNN model for multi-class image classification.
- To train and validate the model using tomato disease images.
- To evaluate the model using accuracy, loss, precision, recall, F1-score, and confusion matrix.
- To develop a simple Streamlit interface for disease prediction.

3. Dataset Description

The tomato leaf disease dataset contains images organized into separate folders according to their respective classes.

The dataset used contains the following **10 classes**:

1. Tomato Bacterial Spot
2. Tomato Early Blight
3. Tomato Healthy
4. Tomato Late Blight
5. Tomato Leaf Mold
6. Tomato Septoria Leaf Spot
7. Tomato Spider Mites
8. Tomato Target Spot
9. Tomato Mosaic Virus
10. Tomato Yellow Leaf Curl Virus

The images are resized to **128 × 128 pixels** before being provided to the CNN.

The dataset is divided into:

- 80% Training data
- 20% Validation data

4. Algorithm

Step 1: Import Required Libraries

Import TensorFlow, NumPy, Matplotlib, and Scikit-learn libraries required for model development and evaluation.

Step 2: Load Dataset

Load the tomato leaf images using `image_dataset_from_directory()`.

The images are automatically assigned labels according to their folder names.

Step 3: Preprocess Images

Resize all images to:

128 × 128 × 3

where 3 represents the RGB color channels.

Pixel values are normalized from the range **0–255 to 0–1**.

Step 4: Data Augmentation

Apply random horizontal flipping, rotation, and zooming to increase the variety of training images and reduce overfitting.

Step 5: Build CNN

The CNN consists of:

- Input layer
- Three convolution layers
- Max pooling layers
- Flatten layer
- Dense layer
- Dropout layer
- Softmax output layer

Step 6: Compile Model

The model is compiled using:

- **Optimizer:** Adam
- **Loss:** Sparse Categorical Crossentropy
- **Metric:** Accuracy

Step 7: Train Model

Train the CNN using the training dataset for a specified number of epochs.

Step 8: Validate Model

Evaluate the trained model using the validation dataset.

Step 9: Generate Predictions

Generate predictions for validation images and calculate:

- Accuracy
- Precision
- Recall
- F1-score

Step 10: Confusion Matrix

Generate a confusion matrix to identify correctly and incorrectly classified disease classes.

Step 11: Test New Image

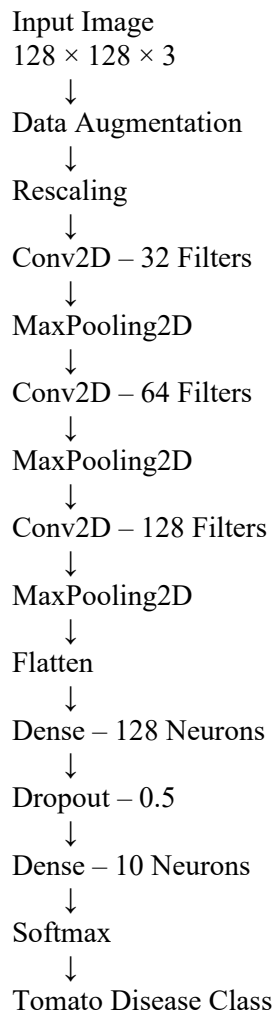
Upload a new tomato leaf image and pass it through the trained CNN to obtain the predicted class and confidence score.

Step 12: Streamlit Application

Load the trained CNN model into a Streamlit application. The application allows the user to upload a tomato leaf image and displays the predicted disease class and confidence.

5. CNN Architecture

The implemented CNN architecture is:



6. Code

```
import os
import numpy as np
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers, models  
from sklearn.metrics import classification_report  
from sklearn.metrics import confusion_matrix
```

```
# Dataset path
```

```
dataset_path = r"C:\Sem5-(TY)\Deep Learning\Deep Learning Labs\Assignment6\tomato\train"
```

```
# Parameters
```

```
IMG_SIZE = (128, 128)
```

```
BATCH_SIZE = 32
```

```
SEED = 42
```

```
# Load training dataset
```

```
train_dataset = tf.keras.utils.image_dataset_from_directory(  
    dataset_path,  
    validation_split=0.2,  
    subset="training",  
    seed=SEED,  
    image_size=IMG_SIZE,  
    batch_size=BATCH_SIZE  
)
```

```
# Load validation dataset
```

```
validation_dataset = tf.keras.utils.image_dataset_from_directory(  
    dataset_path,  
    validation_split=0.2,  
    subset="validation",  
    seed=SEED,  
    image_size=IMG_SIZE,  
    batch_size=BATCH_SIZE  
)
```

```
# Get class names
```

```
class_names = train_dataset.class_names
```

```
print("Classes:")
```

```
for i, name in enumerate(class_names):  
    print(i, name)
```

```
print("Number of Classes:", len(class_names))
```

```
# Data augmentation
```

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1)
])
```

```
# Build CNN
```

```
model = models.Sequential([

    layers.Input(shape=(128, 128, 3)),
    data_augmentation,
    layers.Rescaling(1./255),

    layers.Conv2D(
        32,
        (3, 3),
        activation="relu"
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        128,
        (3, 3),
        activation="relu"
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(
        128,
        activation="relu"
    ),

    layers.Dropout(0.5),

    layers.Dense(
        len(class_names),
        activation="softmax"
```

```
)  
])
```

```
# Display model architecture  
model.summary()
```

```
# Compile model  
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

```
# Train model  
EPOCHS = 15
```

```
history = model.fit(  
    train_dataset,  
    validation_data=validation_dataset,  
    epochs=EPOCHS  
)
```

```
# Evaluate model  
loss, accuracy = model.evaluate(validation_dataset)
```

```
print("Validation Loss:", loss)  
print("Validation Accuracy:", accuracy)
```

```
# Accuracy graph  
plt.figure(figsize=(8, 5))
```

```
plt.plot(  
    history.history["accuracy"],  
    label="Training Accuracy"  
)
```

```
plt.plot(  
    history.history["val_accuracy"],  
    label="Validation Accuracy"  
)
```

```
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.title("Training and Validation Accuracy")
```

```

plt.legend()

plt.show()

# Loss graph
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()

plt.show()

# Generate predictions
y_true = []
y_pred = []

for images, labels in validation_dataset:

    predictions = model.predict(
        images,
        verbose=0
    )

    predicted_labels = np.argmax(
        predictions,
        axis=1
    )

    y_true.extend(labels.numpy())
    y_pred.extend(predicted_labels)

# Classification report
print(
    classification_report(

```

```

        y_true,
        y_pred,
        target_names=class_names
    )
)

# Confusion matrix
cm = confusion_matrix(
    y_true,
    y_pred
)

plt.figure(figsize=(12, 10))
plt.imshow(cm)
plt.title("Confusion Matrix")

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.xticks(
    range(len(class_names)),
    class_names,
    rotation=90
)

plt.yticks(
    range(len(class_names)),
    class_names
)

plt.colorbar()
plt.tight_layout()
plt.show()

# Save trained model
model.save("tomato_disease_cnn.keras")

print("Model saved successfully.")

```

7. Streamlit Application

The trained model is deployed using Streamlit to create an interactive prediction interface.

The application allows the user to:

1. Upload a tomato leaf image.
2. View the uploaded image.

3. Pass the image to the trained CNN.
4. Display the predicted disease.
5. Display the prediction confidence.
6. Display probabilities for all disease classes.

The trained model is saved as:

`tomato_disease_cnn.keras`

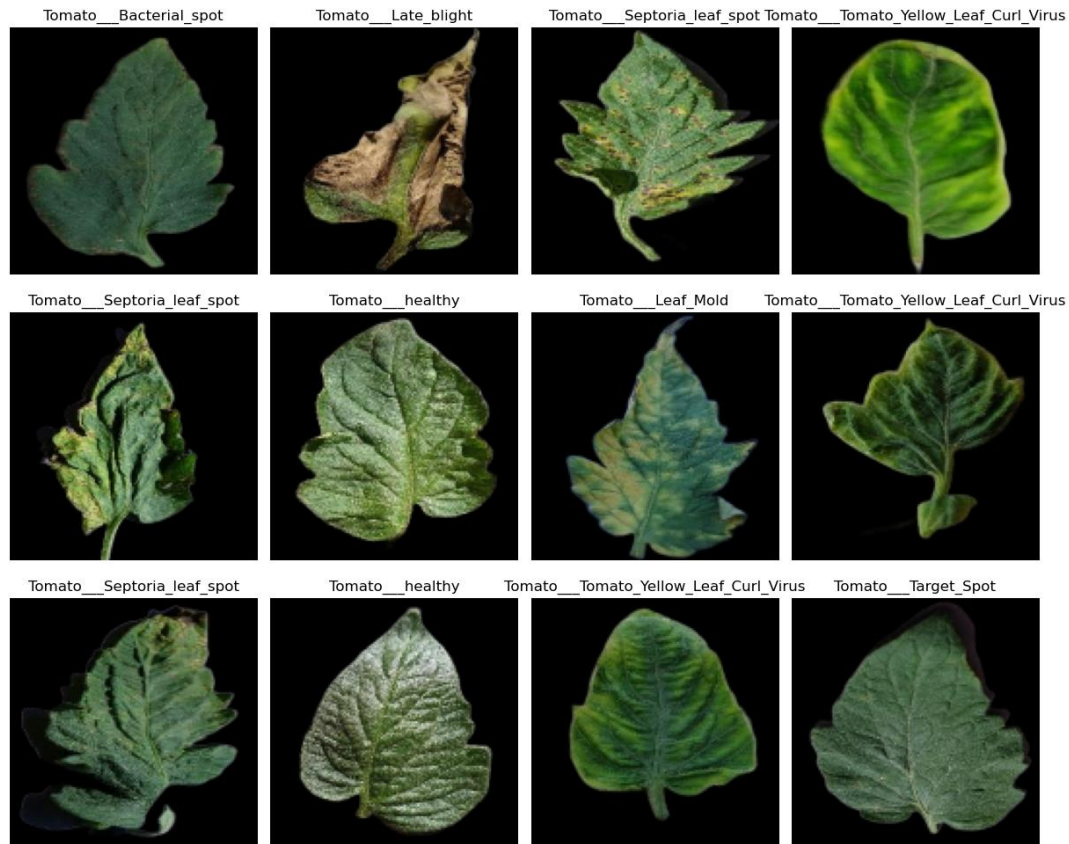
The Streamlit application is implemented in:

`app.py`

8. Output

The following results were obtained from the implementation.

8.1 Dataset Sample Images



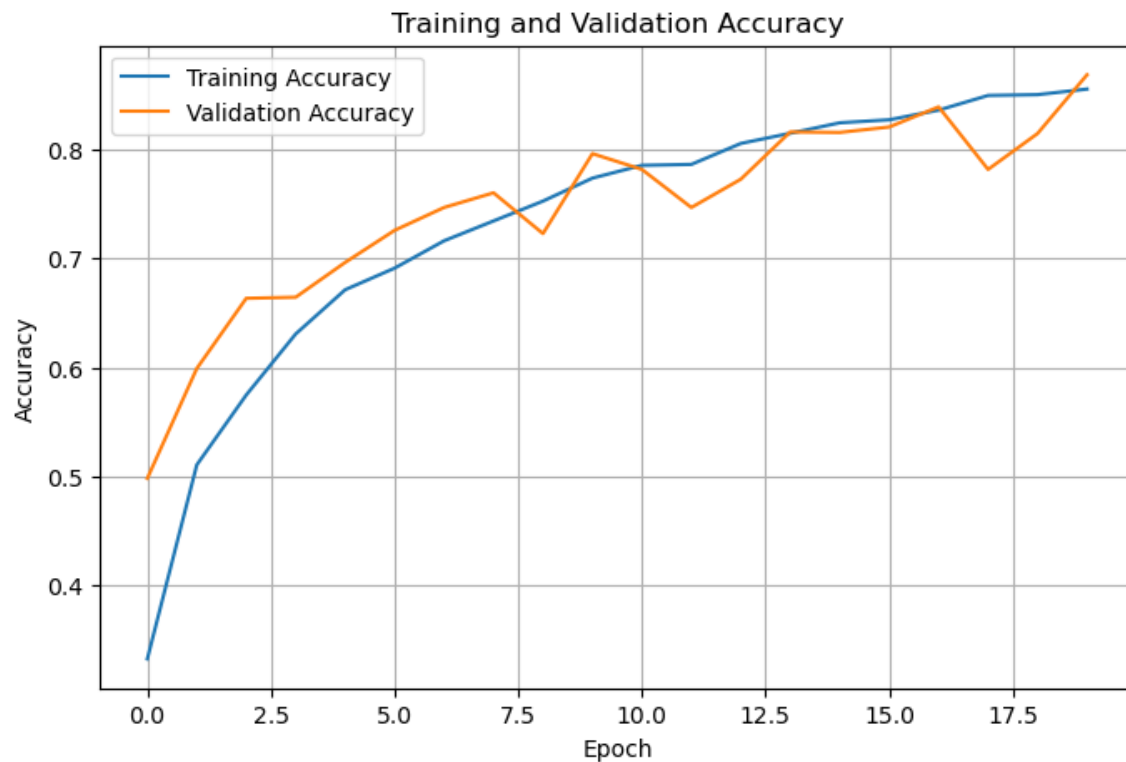
8.2 CNN Model Summary

Layer (type)	Output Shape	Param #
sequential_4 (Sequential)	(None, 128, 128, 3)	0
rescaling_2 (Rescaling)	(None, 128, 128, 3)	0
conv2d_6 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_6 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_7 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_8 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_8 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_2 (Flatten)	(None, 25088)	0
dense_4 (Dense)	(None, 128)	3,211,392
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 10)	1,290

8.3 Training Output

```
Epoch 10/20  
250/250 ————— 91s 365ms/step - accuracy: 0.7740 - loss: 0.6434 - val_accuracy: 0.7965 - val_loss: 0.5607  
Epoch 11/20  
250/250 ————— 82s 326ms/step - accuracy: 0.7857 - loss: 0.6257 - val_accuracy: 0.7820 - val_loss: 0.6175  
Epoch 12/20  
250/250 ————— 80s 320ms/step - accuracy: 0.7866 - loss: 0.6035 - val_accuracy: 0.7470 - val_loss: 0.7474  
Epoch 13/20  
250/250 ————— 84s 336ms/step - accuracy: 0.8059 - loss: 0.5619 - val_accuracy: 0.7730 - val_loss: 0.6847  
Epoch 14/20  
250/250 ————— 80s 321ms/step - accuracy: 0.8152 - loss: 0.5297 - val_accuracy: 0.8165 - val_loss: 0.5617  
Epoch 15/20  
250/250 ————— 77s 307ms/step - accuracy: 0.8249 - loss: 0.4989 - val_accuracy: 0.8160 - val_loss: 0.5760  
Epoch 16/20  
250/250 ————— 77s 306ms/step - accuracy: 0.8276 - loss: 0.4916 - val_accuracy: 0.8210 - val_loss: 0.5399  
Epoch 17/20  
250/250 ————— 82s 329ms/step - accuracy: 0.8365 - loss: 0.4657 - val_accuracy: 0.8395 - val_loss: 0.4787  
Epoch 18/20  
250/250 ————— 80s 320ms/step - accuracy: 0.8500 - loss: 0.4565 - val_accuracy: 0.7820 - val_loss: 0.7240  
Epoch 19/20  
250/250 ————— 81s 326ms/step - accuracy: 0.8508 - loss: 0.4393 - val_accuracy: 0.8150 - val_loss: 0.5567  
Epoch 20/20  
250/250 ————— 78s 313ms/step - accuracy: 0.8559 - loss: 0.4164 - val_accuracy: 0.8690 - val_loss: 0.3571
```

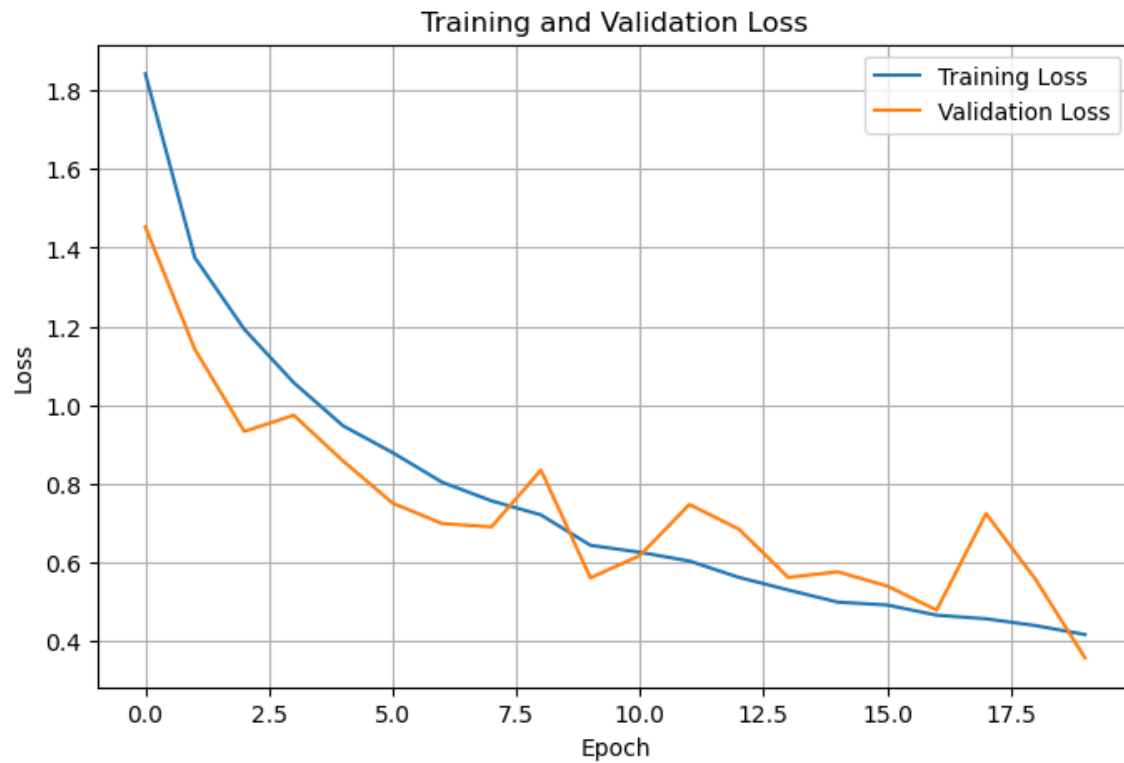
8.4 Training and Validation Accuracy



Observation:

The graph shows the variation of training and validation accuracy across the training epochs.

8.5 Training and Validation Loss



Observation:

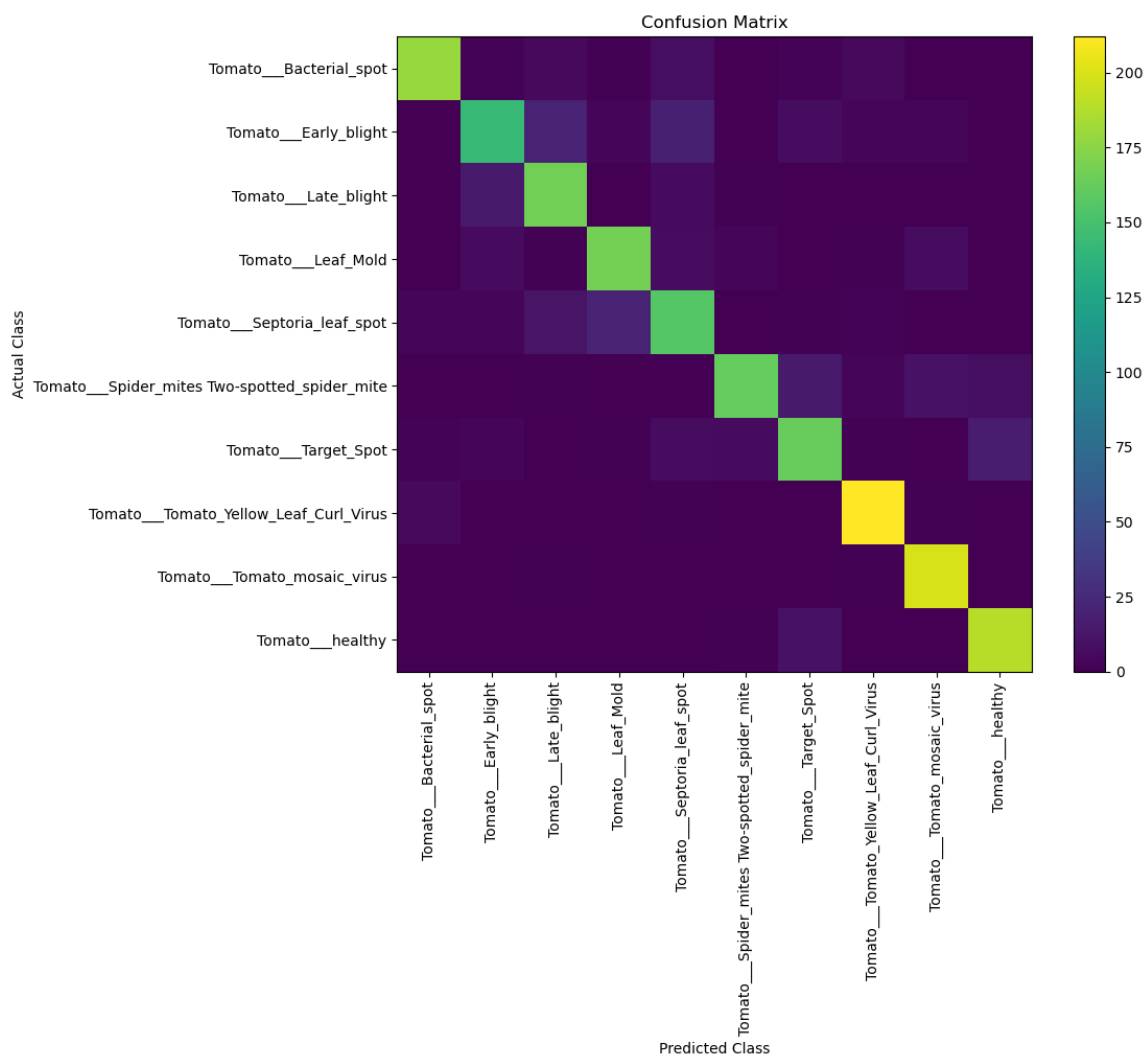
The loss generally decreases as the model learns features from the training images.

8.6 Classification Report

	precision	recall	f1-score	support
Tomato__Bacterial_spot	0.95	0.89	0.92	202
Tomato__Early_blight	0.83	0.72	0.77	198
Tomato__Late_blight	0.81	0.88	0.85	189
Tomato__Leaf_Mold	0.87	0.87	0.87	194
Tomato__Septoria_leaf_spot	0.76	0.79	0.78	197
Tomato__Spider_mites Two-spotted_spider_mite	0.93	0.81	0.87	200
Tomato__Target_Spot	0.82	0.81	0.82	200
Tomato__Tomato_Yellow_Leaf_Curl_Virus	0.93	0.97	0.95	219
Tomato__Tomato_mosaic_virus	0.90	0.99	0.95	201
Tomato__healthy	0.88	0.94	0.91	200
accuracy			0.87	2000
macro avg	0.87	0.87	0.87	2000
weighted avg	0.87	0.87	0.87	2000

The classification report provides precision, recall, F1-score, and support for each tomato disease class.

8.7 Confusion Matrix



The confusion matrix shows the number of correct and incorrect predictions for each disease class.

8.8 Streamlit Web Application

Tomato Disease Detector

Upload an image of a tomato leaf and the trained CNN will predict the disease class.

 Upload a tomato leaf image



5c8c542...d 8628.JPG
20.0KB



Uploaded Image



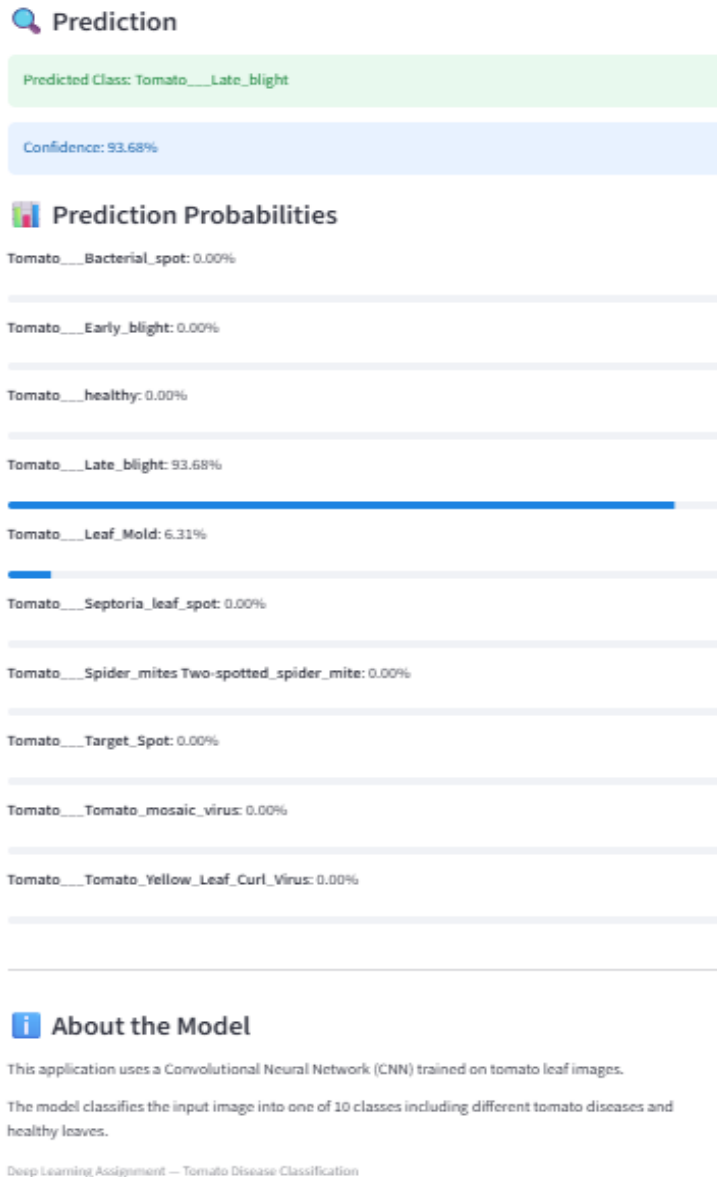
Tomato Leaf

Prediction

Predicted Class: Tomato__Late_blight

Confidence: 93.68%

Prediction Probabilities



The application accepts a tomato leaf image and displays the predicted disease class along with its confidence score.

9. Result

The CNN model was successfully designed and trained for multi-class tomato leaf disease classification. The model was able to learn visual features from tomato leaf images and classify them into the corresponding disease categories.

The model performance was evaluated using validation accuracy, loss, precision, recall, F1-score, and a confusion matrix. A Streamlit-based web application was also successfully developed to provide real-time disease prediction from an uploaded tomato leaf image.

10. Conclusion

A Convolutional Neural Network was successfully implemented for tomato disease image classification. The convolution and pooling layers extracted important visual features from the leaf images, while the dense and softmax layers performed the final classification. Data augmentation and dropout were used to improve the model's ability to generalize and reduce overfitting. The trained model was integrated with a Streamlit web application, providing a simple interface for users to upload tomato leaf images and obtain disease predictions. The project demonstrates the practical application of Deep Learning, CNN-based image classification, model evaluation, and deployment using Streamlit.